

PRD v2.0 · REFINED

SquadSync

VERSION

2.0 — Refined & Implementation-Ready

STATUS

APPROVED FOR DEVELOPMENT

TARGET LAUNCH

Q3 2026 · 6-Day Sprint

STACK

Next.js 15 · MongoDB · Pusher

DEPLOYMENT

Vercel + GitHub Actions

AUTHOR

SquadSync Engineering Team

Real-Time Matchmaking

Snake Draft Balancing

Karma Reputation

Turf Coordination

Live Chat

Table of Contents

1.	Executive Summary	3
2.	Problem Statement	3
3.	User Personas	3
4.	Core Vision & MVP Feature Scope	4
5.	Functional Requirements	4
6.	Non-Functional Requirements	5
7.	Technical Architecture	6
8.	Realtime Communication: Pusher vs Socket.IO	6
9.	Database Schema Overview	7
10.	API & Module Breakdown	9
11.	Suggested Folder Architecture	10
12.	State Management & Auth Strategy	11
13.	Team Balancing Algorithm	11
14.	Karma Engine	12
15.	6-Day Development Roadmap	12
16.	Risk Register & Mitigation	14
17.	Success Metrics	14
18.	Future Enhancements	15

1 Executive Summary

SquadSync is a full-stack, real-time matchmaking and reputation platform designed for amateur 5v5 football. It solves the three core pain points of amateur football — **finding players, ensuring balance, and preventing no-shows** — through an intelligent matchmaking queue, a Snake Draft team balancer, and a Karma reputation engine.

Built on Next.js 15, MongoDB Atlas, and Pusher, SquadSync is architected for serverless deployment on Vercel with a 6-day development sprint in mind. The system prioritizes reliability, real-time responsiveness, and a polished user experience over enterprise-scale complexity.

2 Problem Statement

Amateur football organizers and players consistently face three structural problems:

Player Density	Finding exactly 10 available players in a specific locality at the same time is logistically difficult without a dedicated coordination layer.
Match Imbalance	Games are frequently lopsided because there is no data-driven mechanism to split players by skill level, position, or experience.
The No-Show Problem	Players commit to matches and fail to appear, with no accountability mechanism, degrading the experience for the nine other participants.

3 User Personas

Persona	Profile	Primary Goal	Key Pain Point
The Lone Wolf	Individual player, new to city or college. No existing football network.	Join a game quickly without needing a pre-formed group.	Social friction: doesn't know anyone to play with.
The Captain	Regular organizer. Highly competitive, values match quality over convenience.	Fill a reliable 10-player lobby with balanced teams and zero no-shows.	Wasted time managing cancellations and unbalanced squads.
The Casual	Plays occasionally. Low commitment, high interest in fun over competition.	Join a nearby game with minimal setup friction.	Over-complicated booking flows that deter casual participation.

The Turf Owner	Manages one or more local football grounds. Wants consistent bookings.	Maximize slot utilization and deal with organized, pre-coordinated groups.	Empty slots from last-minute cancellations and disorganized bookings.
----------------	--	--	---

4

Core Vision & MVP Feature Scope

The MVP must demonstrate all five pillars of the platform. Features beyond this scope are classified as **Post-MVP** and should not be attempted within the 6-day sprint.

Feature	Priority	Sprint Inclusion	Notes
Auth (Google OAuth via NextAuth.js)	P0 — Critical	✓ MVP	
Player Profile & Match Profile Setup	P0 — Critical	✓ MVP	Position + 5 attributes
Matchmaking Queue (Region-Based)	P0 — Critical	✓ MVP	10-player lobby fill
Real-Time Lobby (Pusher Presence Channel)	P0 — Critical	✓ MVP	Live join/leave/ready
Snake Draft Team Balancing	P0 — Critical	✓ MVP	Auto team split on fill
In-Lobby Real-Time Chat	P1 — High	✓ MVP	Ephemeral, lobby-scoped
Turf Voting (Majority 6/10)	P1 — High	✓ MVP	Pre-seeded turf data
Captain Auto-Selection	P1 — High	✓ MVP	Highest Karma in each team
Post-Match Score Reporting	P1 — High	✓ MVP	Dual captain report
Peer Ratings (Thumbs Up / Down)	P1 — High	✓ MVP	Post-match only
Karma Score Engine	P1 — High	✓ MVP	Auto-recalculated
Shadow / Hard Ban System	P2 — Medium	✓ MVP	Threshold-based
Turf Owner Dashboard	P2 — Medium	Post-MVP	Future sprint
Push Notifications (Browser)	P2 — Medium	Post-MVP	Nice to have
Geospatial Map View	P3 — Low	Post-MVP	Future enhancement
Payment Integration	P3 — Low	Post-MVP	Out of scope

5

Functional Requirements

5.1 Identity & Profile Management

- FR-01** Users authenticate via Google OAuth 2.0 using NextAuth.js. Email/password auth is out of scope for the MVP.
- FR-02** On first login, users complete an onboarding flow to set their **Match Profile**: preferred position (GK, DEF, MID, FWD) and five skill attributes (Pace, Shooting, Passing, Defending, Physical), each rated 40–100.
- FR-03** The system maintains a **Karma Score** (0–100, default 70) for every user, recalculated after each completed match.
- FR-04** Users with Karma below 40 enter **Shadow Ban** status — they can still join lobbies but are ranked last in matchmaking. Users below 20 receive a **Hard Ban** and are blocked from all queue actions.

5.2 Matchmaking & Lobbies

- FR-05** A player enters the matchmaking queue by clicking **Find Game**. The system places them in an active lobby for their region, or creates a new one.
- FR-06** Lobbies hold up to 10 players. Each player must click **Ready Up** to confirm participation. All 10 must be ready for the lobby to advance.
- FR-07** Lobbies that do not fill within **10 minutes** are auto-expired via a MongoDB TTL index on the *expiresAt* field. Players are notified via Pusher.
- FR-08** When all 10 players are ready, the system automatically runs the **Snake Draft Algorithm** to split players into Team A and Team B. No manual team selection is permitted.
- FR-09** Real-time lobby state (players joining/leaving/readying) is broadcast to all members via a Pusher **presence channel** (*presence-lobby-{id}*).

5.3 Pre-Match Coordination

- FR-10** Captain selection is automatic: the player with the highest Karma Score in each team is assigned as captain. No voting required.
- FR-11** The lobby enters a **Turf Voting phase**: each player votes for one of 3 pre-seeded nearby turfs. The turf with the most votes (simple plurality) is selected after all votes are cast or a 2-minute timer expires.
- FR-12** An ephemeral real-time **Lobby Chat** is available from lobby creation until the match is marked as completed. Chat history is not persisted beyond 24 hours.

- FR-13** **Players** receive an in-app notification with match details: Turf name, address, scheduled time, team composition, and captain contact.

5.4 Post-Match & Reputation

- FR-14** **Score Reporting:** Both captains independently submit the final score. If both reports agree, the match is confirmed automatically. If they disagree by more than 1 goal, the match is flagged as **Disputed** and both scores are recorded for manual review.
- FR-15** **Peer Rating:** After a match, each player can rate every teammate and opponent with a Thumbs Up or Thumbs Down. Ratings are anonymous and contribute to the Karma Score recalculation.
- FR-16** **No-Show Detection:** If a player does not confirm attendance within 30 minutes of match start time, they are flagged as a no-show. Karma is deducted automatically (–15 points).

6 Non-Functional Requirements

Category	Requirement	Target / Implementation
Performance	Real-time lobby events (join, ready, chat, vote) must reach clients with minimal delay.	< 200ms via Pusher managed WebSockets
Scalability	Lobby channels must be region-scoped to prevent cross-lobby data leakage.	Pusher presence channels per lobby ID
Availability	Auth and matchmaking endpoints must remain available during deployment.	Vercel serverless ~99.5% SLA; graceful error states in UI
Security	All API secrets (Pusher keys, MongoDB URI, NextAuth secret) must use environment variables. Pusher private/presence channels require server-side auth.	env vars + /api/pusher/auth endpoint
Data Integrity	Lobby slot conflicts (two users joining the 10th slot simultaneously) must be prevented.	MongoDB atomic \$push with \$size guard or optimistic locking
UX Responsiveness	All page transitions and lobby state updates must feel instantaneous to the user.	Optimistic UI updates + Pusher reconciliation
Code Quality	TypeScript strict mode throughout; no any types in production code.	ESLint + TypeScript strict

7 Technical Architecture

7.1 Recommended Tech Stack

Layer	Technology	Rationale
Frontend	Next.js 15 (App Router) + TypeScript	Server Components for SEO pages; Client Components for real-time UI
Styling	Tailwind CSS + Shadcn/UI	Rapid, consistent UI without custom CSS overhead
Backend	Next.js API Routes (not Server Actions for real-time)	API routes support Pusher auth triggers; Server Actions are for mutations only
Database	MongoDB Atlas + Mongoose	Flexible schema for evolving lobby states; native TTL indexing; 2dsphere for geo

Real-Time	Pusher (Channels)	Purpose-built for serverless; see Section 8 for full comparison
Auth	NextAuth.js v5 (Google Provider)	Fastest OAuth setup; JWT sessions; built-in CSRF protection
State Management	Zustand	Lightweight, no boilerplate; ideal for lobby + match real-time state
Deployment	Vercel + GitHub Actions	Zero-config Next.js deployment; preview URLs per PR
CI/CD	GitHub Actions	Auto-lint, type-check, and deploy on push to main

**ARCH
NOTE**

Why API Routes, not Server Actions for real-time? Server Actions are optimized for form mutations and do not expose a stable HTTP endpoint. Pusher's auth endpoint requires a POST handler at a fixed URL (*/api/pusher/auth*). Similarly, triggering Pusher events from lobby join/leave logic must happen in an API route that returns a JSON response. Use Server Actions only for profile saves and non-real-time mutations.

8

Realtime Communication: Pusher vs Socket.IO

The original PRD listed both Pusher and WebSockets/Socket.IO. This section provides a definitive recommendation for this project's constraints.

Factor	Pusher (Recommended)	Socket.IO
Vercel Compatibility	Native — works with serverless functions	Requires persistent server (Render, Railway, or separate VPS)
Setup Complexity	~30 min to working presence channel	2–4 hours: separate Express server + CORS config
Presence Channels	Built-in: who's online in lobby, member list	Must implement manually with Redis adapter
Free Tier	200 concurrent connections, 200k messages/day	Self-hosted — no limits, but you pay for server
Auth for Private Channels	Single /api/pusher/auth endpoint	Custom middleware per namespace
Reliability	Managed, 99.999% SLA, global CDN	Depends on your hosting tier
Best For	Serverless (Vercel) student projects	Enterprise apps with persistent infra

VERDICT

Use Pusher. Socket.IO is architecturally incompatible with Vercel's serverless model without a dedicated persistent server. For a 6-day student sprint deploying to Vercel, Pusher eliminates 4–8 hours of infrastructure setup and provides a superior developer experience. Pusher's free tier comfortably supports demo and evaluation scale. The Pusher SDK is also significantly simpler to integrate within Next.js.

8.1 Pusher Channel Strategy

Channel Name	Type	Purpose	Events
presence-lobby-{lobbyId}	Presence	Live player list, ready states, votes, chat	player-joined, player-left, player-ready, chat-message, teams-formed, vote-cast
private-match-{matchId}	Private	Post-match chat continuation, score reporting	score-submitted, match-confirmed, match-disputed

public-region-{regionCode}	Public	Notify players in a region of new lobbies	lobby-created, lobby-available
----------------------------	--------	---	--------------------------------

9 Database Schema Overview

Four primary MongoDB collections. Indexes are critical — define them in *lib/db/models/* before writing business logic.

9.1 Users Collection

Field	Type	Notes
_id	ObjectId	Auto-generated
name	String	From OAuth provider
email	String	Unique index. From OAuth provider
image	String	Avatar URL from OAuth
position	Enum	'GK' 'DEF' 'MID' 'FWD'
attributes.pace	Number	40–100. Default 60
attributes.shooting	Number	40–100
attributes.passing	Number	40–100
attributes.defending	Number	40–100
attributes.physical	Number	40–100
karmaScore	Number	0–100. Default 70. Index for matchmaking sort
matchesPlayed	Number	Incremented on match confirm
matchesCompleted	Number	Incremented on attendance confirm
noShows	Number	Incremented on no-show detection
isBanned	Boolean	Derived from karmaScore thresholds
banType	Enum null	'shadow' 'hard' null
onboardingComplete	Boolean	Gate: must be true to access queue
createdAt	Date	Auto timestamp

9.2 Lobbies Collection

Field	Type	Notes
_id	ObjectId	Also used as Pusher channel suffix
status	Enum	'waiting' 'ready_check' 'voting' 'confirmed' 'active' 'completed' 'expired'
region	String	ISO city/region code. Index
location	GeoJSON Point	{ type: 'Point', coordinates: [lng, lat] }. 2dsphere index
players	Array	{ userId, joinedAt, isReady, team: 'A' 'B' null }
teamA	Array	Populated after Snake Draft
teamB	Array	Populated after Snake Draft
captainA	ObjectId null	Auto-assigned: highest Karma in teamA
captainB	ObjectId null	Auto-assigned: highest Karma in teamB
turfVotes	Array	{ userId, turfId }
selectedTurf	ObjectId null	Set after voting resolves
scheduledAt	Date null	Confirmed match start time
expiresAt	Date	TTL index. Set to createdAt + 10 min when status='waiting'
createdAt	Date	Auto timestamp

IMPORTANT

The TTL index must be set on **expiresAt**, not *createdAt*. Only lobbies in 'waiting' status should have this field set. Once a lobby advances to 'voting' or beyond, remove *expiresAt* to prevent premature deletion. Use `mongoose.model().updateOne({ $unset: { expiresAt: 1 } })`.

9.3 Matches Collection

Field	Type	Notes
_id	ObjectId	
lobbyId	ObjectId	Ref: Lobbies
teamA	Array	Snapshot from lobby at match start
teamB	Array	Snapshot from lobby at match start
captainA	ObjectId	
captainB	ObjectId	
scoreReportA	Object null	{ goalsA, goalsB, submittedAt }
scoreReportB	Object null	{ goalsA, goalsB, submittedAt }
finalScore	Object null	{ teamA: Number, teamB: Number }
status	Enum	'pending_reports' 'confirmed' 'disputed'
playerRatings	Array	{ raterId, ratedId, thumbsUp: Boolean }
attendanceConfirmed	Array	Players who confirmed attendance
createdAt	Date	
completedAt	Date null	Set on status = confirmed

9.4 Turfs Collection (Seed Data)

Turf data is pre-seeded for demo purposes. A Turf Owner management portal is a post-MVP feature.

Field	Type	Notes
_id	ObjectId	
name	String	e.g. 'Green Turf Arena'
address	String	Human-readable address
location	GeoJSON Point	For proximity queries. 2dsphere index

slots	Array	{ date, time, isBooked, bookedByLobbyId }
pricePerHour	Number	In local currency
contactNumber	String	Turf owner contact
images	Array	Image URLs
region	String	Matches lobby region codes

10

API & Module Breakdown

Method	Endpoint	Description	Auth
POST	/api/auth/[...nextauth]	NextAuth.js handler (OAuth flow)	Public
GET	/api/users/me	Fetch current user profile	Required
PUT	/api/users/me	Update Match Profile (attributes, position)	Required
GET	/api/lobbies	List open lobbies for the user's region	Required
POST	/api/lobbies	Create a new matchmaking lobby	Required
GET	/api/lobbies/[id]	Fetch lobby state + players	Required
POST	/api/lobbies/[id]/join	Join a lobby (atomic, max 10)	Required
POST	/api/lobbies/[id]/leave	Leave a lobby	Required
POST	/api/lobbies/[id]/ready	Toggle player ready status	Required
POST	/api/lobbies/[id]/vote/turf	Submit turf vote	Required
GET	/api/turfs	Get nearby turfs by region or coordinates	Required
POST	/api/matches/[id]/report	Captain submits score report	Captain only
POST	/api/matches/[id]/rate	Submit peer ratings for all players	Required
POST	/api/matches/[id]/attend	Confirm physical attendance	Required
POST	/api/pusher/auth	Authenticate private/presence channel subscriptions	Required

11 Suggested Folder Architecture

Next.js 15 App Router structure. All real-time logic lives in `/hooks` and `/lib/pusher.ts`. Business logic (algorithms) is isolated in `/lib/utlils/` for easy testing.

squadsync/		■ ■ ■■ LobbyChat.tsx	
■ ■ ■■ app/		■ ■ ■■ match/	
■ ■ ■■ (auth)/		■ ■ ■■ ScoreReportForm.tsx	
■ ■ ■■ login/page.tsx		■ ■ ■■ PlayerRatingCard.tsx	
■ ■ ■■ onboarding/page.tsx	← Match	■ ■ ■■ profile/	
Profile setup		■ ■ ■■ KarmaBadge.tsx	
■ ■ ■■ (app)/		■ ■ ■■ AttributeRadar.tsx	
■ ■ ■■ dashboard/page.tsx	← Find	■ ■ ■■ hooks/	
Game, active lobbies		■ ■ ■■ useLobby.ts	← Push
■ ■ ■■ lobby/[id]/page.tsx	← Liv	er subscription + state	
e lobby view		■ ■ ■■ usePresence.ts	← Onli
■ ■ ■■ match/[id]/page.tsx	← Pos	ne members in lobby	
t-match flow		■ ■ ■■ useMatch.ts	
■ ■ ■■ profile/page.tsx	← Kar	■ ■ ■■ lib/	
ma, stats, history		■ ■ ■■ db/	
■ ■ ■■ api/		■ ■ ■■ connect.ts	← Mon
■ ■ ■■ auth/[...nextauth]/route.ts		goose connection (cached)	
■ ■ ■■ users/me/route.ts		■ ■ ■■ models/	
■ ■ ■■ lobbies/		■ ■ ■■ User.ts	
■ ■ ■■ route.ts	← GET	■ ■ ■■ Lobby.ts	
list, POST create		■ ■ ■■ Match.ts	
■ ■ ■■ [id]/		■ ■ ■■ Turf.ts	
■ ■ ■■ route.ts	← GET	■ ■ ■■ pusher.ts	← Serv
state		er + client Pusher instances	
■ ■ ■■ join/route.ts		■ ■ ■■ auth.ts	← Next
■ ■ ■■ leave/route.ts		Auth config	
■ ■ ■■ ready/route.ts		■ ■ ■■ utils/	
■ ■ ■■ vote/turf/route.ts		■ ■ ■■ teamBalancer.ts	← Snak
■ ■ ■■ matches/[id]/		e Draft Algorithm	
■ ■ ■■ report/route.ts		■ ■ ■■ karmaEngine.ts	← Karm
■ ■ ■■ rate/route.ts		a recalculation	
■ ■ ■■ attend/route.ts		■ ■ ■■ store/	
■ ■ ■■ turfs/route.ts		■ ■ ■■ lobbyStore.ts	← Zust
■ ■ ■■ pusher/auth/route.ts	← CRI	and: lobby + match state	
TICAL: Pusher channel auth		■ ■ ■■ types/	
■ ■ ■■ layout.tsx		■ ■ ■■ index.ts	← Shar
■ ■ ■■ page.tsx	← Land	ed TS interfaces	
ing / redirect		■ ■ ■■ .env.local	← Secr
■ ■ ■■ components/		ets (never commit)	
■ ■ ■■ ui/	← Shad	■ ■ ■■ next.config.ts	
cn/UI components			
■ ■ ■■ lobby/			
■ ■ ■■ LobbyCard.tsx			
■ ■ ■■ PlayerSlot.tsx			
■ ■ ■■ TeamGrid.tsx			

12

State Management & Authentication Strategy

12.1 Authentication — NextAuth.js v5

Use **NextAuth.js v5** with the Google OAuth provider. This provides one-click sign-in with zero password management overhead — ideal for a student project. Configure a JWT session strategy (no database sessions needed). Store *userId* in the JWT token for fast API authentication via *getSession()* in API routes.

ENV VARS

NEXTAUTH_SECRET, NEXTAUTH_URL, GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, MONGODB_URI, PUSHER_APP_ID, PUSHER_KEY, PUSHER_SECRET, PUSHER_CLUSTER, NEXT_PUBLIC_PUSHER_KEY, NEXT_PUBLIC_PUSHER_CLUSTER

12.2 State Management — Zustand

Use **Zustand** for global client-side state. Two primary stores are needed:

Store	State Held	Updated By
lobbyStore	Current lobby players, ready states, team assignments, vote results, chat messages	Pusher event handlers in useLobby.ts
userStore	Authenticated user profile, Karma score, onboarding status	NextAuth session on mount; API mutations

13

Team Balancing — Snake Draft Algorithm

The Snake Draft is a proven, fair-allocation algorithm used in real sports drafts. It guarantees that neither team consistently gets an advantage from pick order. Implementation lives in *lib/utils/teamBalancer.ts*.

Step 1: Compute Power Score for each player

FORMULA

$$\text{powerScore} = (\text{pace} \times 0.2 + \text{shooting} \times 0.2 + \text{passing} \times 0.2 + \text{defending} \times 0.2 + \text{physical} \times 0.2) \times (0.7 + \text{karmaScore} / 100 \times 0.3)$$

The Karma multiplier (0.7–1.0) rewards reliable players with a slight power boost without making it the dominant factor. A player with all-60 attributes and 70 Karma has $\text{powerScore} \approx 63$. The same player with 100 Karma scores ≈ 66 .

Step 2: Sort all 10 players by powerScore (descending)

Step 3: Apply Snake Draft pick order

Pick #	1	2	3	4	5	6	7	8	9	10
Team	A	B	B	A	A	B	B	A	A	B

Team A picks: 1st, 4th, 5th, 8th, 9th. Team B picks: 2nd, 3rd, 6th, 7th, 10th. The snake pattern ensures that neither team gets a run of top picks. Verify balance after assignment: $|avgPowerA - avgPowerB|$ should be $<10\%$.

14

Karma Engine

Karma is recalculated after every completed match. The engine runs server-side in `lib/utls/karmaEngine.ts`, triggered by the match confirmation webhook.

FORMULA	$newKarma = clamp(currentKarma + (matchCompleted ? +3 : 0) + (noShow ? -15 : 0) + (positiveRatingsReceived \times 1) - (negativeRatingsReceived \times 2) , 0, 100)$ Negative ratings are penalized twice as heavily as positives to deter chronic bad actors faster than they can recover.
---------	--

Karma Range	Status	Effect
71–100	Trusted	Priority matchmaking placement; green badge
41–70	Standard	Normal matchmaking queue; yellow/grey badge
21–40	Shadow Banned	Can join lobbies but ranked last; warning badge shown only to self
0–20	Hard Banned	Blocked from all matchmaking actions; shown ban reason and appeal process

15

6-Day Development Roadmap

Tasks are ordered by dependency. Complete each day's deliverables before moving forward — each day builds on the previous. Aim for a working vertical slice by end of Day 3 (a player can find and join a game).

Day 1 Foundation & Authentication

- ✓ Initialize Next.js 15 project with TypeScript, Tailwind, Shadcn/UI
 - ✓ Configure MongoDB Atlas connection with Mongoose (cached client in lib/db/connect.ts)
 - ✓ Set up NextAuth.js with Google OAuth; protect routes with middleware
 - ✓ Create User Mongoose model with all fields and indexes
 - ✓ Build Onboarding flow: position picker + attribute sliders → saves to DB
 - ✓ Deploy to Vercel; configure all environment variables
- ★ Deliverable: A user can sign in with Google, complete their profile, and see a dashboard

Day 2 Matchmaking Queue & Lobby Core

- ✓ Create Lobby Mongoose model with TTL index on expiresAt
 - ✓ Build POST /api/lobbies (create) and GET /api/lobbies (list by region)
 - ✓ Build POST /api/lobbies/[id]/join with atomic player slot guard (max 10)
 - ✓ Set up Pusher server instance and /api/pusher/auth endpoint
 - ✓ Implement useLobby.ts hook: subscribe to presence-lobby-{id} channel
 - ✓ Build LobbyCard and PlayerSlot components; render live presence list
- ★ Deliverable: A player can create or join a lobby and see others join in real-time

Day 3 Ready System & Team Balancing

- ✓ Implement POST /api/lobbies/[id]/ready — toggle ready state, broadcast via Pusher
- ✓ Lobby watcher: when all 10 players ready, auto-trigger team balancing
- ✓ Implement teamBalancer.ts: powerScore calc + Snake Draft algorithm
- ✓ Update lobby document: set teamA, teamB, captainA, captainB; advance status to 'voting'

- ✓ Broadcast teams-formed Pusher event; update UI to show team splits

★ **Deliverable: A 10-player lobby automatically splits into balanced teams**

Day 4 Turf Voting & Chat

- ✓ Seed Turf collection with 5–10 local turfs per region (use a seed script)
- ✓ Build POST /api/lobbies/[id]/vote/turf; broadcast vote-cast event via Pusher
- ✓ Implement voting resolution: plurality winner after all votes cast or 2-min timer
- ✓ Build LobbyChat.tsx: optimistic message send + real-time receive via Pusher
- ✓ Implement chat-message event in useLobby.ts; display ephemeral message list
- ✓ Build Match Confirmation screen with turf details, team list, captain info

★ **Deliverable: Players vote for a turf, chat in real-time, see confirmed match details**

Day 5 Post-Match Flow & Karma

- ✓ Create Match document on lobby confirmation; link lobbyId
- ✓ Build POST /api/matches/[id]/report: dual captain score submission with conflict detection
- ✓ Implement score comparison logic: agree → confirmed; disagree >1 goal → disputed
- ✓ Build POST /api/matches/[id]/rate: peer rating submission (thumbs up/down)
- ✓ Implement karmaEngine.ts: recalculate and persist Karma after match confirm
- ✓ Build Profile page: show Karma score, badge, match history, stats

★ **Deliverable: Full post-match flow works; Karma updates after match completion**

Day 6 Polish, Testing & Deployment

- ✓ Fix all TypeScript errors (strict mode); remove any any types
- ✓ Add loading states, error boundaries, and empty state UI throughout
- ✓ Mobile-responsive audit: test on 375px viewport
- ✓ End-to-end test the full user journey (sign-in → lobby → match → karma)
- ✓ Add seed data script for demo (multiple users with varied Karma scores)
- ✓ Final Vercel deployment; test all env variables in production

✓ Prepare demo script: pre-populate demo accounts for live presentation

★ **Deliverable: Production-ready app, demo-ready with seed data**

16 Risk Register & Mitigation

Risk	Likelihood	Impact	Mitigation Strategy
Low user density in demo — can't fill a 10-player lobby organically	High	High	Pre-seed 9 bot/demo accounts with varied Karma. Create a 'Fill with Bots' dev mode button. For demos, use 2 real accounts + 8 seeds.
Pusher free tier limit (200 concurrent connections) hit during demo	Low	Medium	Free tier is more than sufficient for demo scale. Monitor in Pusher dashboard. Upgrade to hobby plan (\$9/mo) if needed.
MongoDB cold start latency on Vercel serverless functions	High	Low	Cache the Mongoose connection across invocations using a module-level variable (standard Next.js pattern). Connection reuse eliminates cold-start overhead.
Race condition: two users claiming the 10th lobby slot simultaneously	Medium	High	Use MongoDB's atomic \$push with a \$size-guarded condition: { \$where: 'this.players.length < 10' }. Return 409 if guard fails.
Score dispute deadlock — captains disagree, no resolution	Medium	Low	Auto-accept if scores differ by exactly 1 goal (rounding). Flag as Disputed otherwise. No manual admin needed for MVP — disputed matches simply don't affect Karma.
Vercel deployment failing due to missing env variables	Medium	High	Maintain a .env.example file in the repo. Add a startup check in next.config.ts that throws if critical env vars are undefined.
Pusher auth endpoint not configured — presence channels silently fail	High	High	Test /api/pusher/auth endpoint independently before building lobby UI. This is the #1 source of real-time bugs in Pusher + Next.js projects.

17 Success Metrics

Metric	Definition	Target
Time-to-Match	Median time from clicking 'Find Game' to all 10 players being ready	< 3 min (demo with seeds)

Balance Accuracy	Absolute difference between Team A and Team B average Power Score	< 10% difference
Reliability Rate	Percentage of confirmed matches completed without a recorded no-show	> 80% (MVQ quality bar)
Real-Time Latency	Time between a Pusher event trigger and client-side UI update	< 200ms p95
Onboarding Completion	Percentage of registered users who complete the Match Profile setup	> 85%
Karma Accuracy	Karma score correctly reflects a player's last 5 matches of behavior	100% (automated calculation)

18

Future Enhancements (Post-MVP)

Enhancement	Value	Complexity	Phase
Turf Owner Dashboard (slot management, booking calendar)	Enables real commercial partnerships	Medium	Phase 2
Browser Push Notifications (Web Push API)	Re-engages users when a lobby fills near them	Medium	Phase 2
Geospatial Map View (nearby lobbies on map)	Dramatically improves discoverability	Medium	Phase 2
ELO-style Skill Rating (replaces attribute self-report)	Fairer, data-driven skill assessment over time	High	Phase 3
Mobile App (React Native + Expo)	80%+ of use case is mobile-first	High	Phase 3
In-App Payments (split turf cost via Stripe)	Removes the biggest logistical friction point	High	Phase 3
Seasonal Leagues & Leaderboards	Long-term retention; competitive motivation	Medium	Phase 3
AI Match Commentary (post-match summary)	Unique feature; high shareability	Low-Medium	Phase 4
Video Highlight Upload (match clips)	Community building; social layer	High	Phase 4

**FINAL
NOTE**

This PRD is scoped for a **6-day student full-stack sprint**. It is intentionally ambitious but achievable. The MVP scope (Sections 4–14) covers everything needed for a technically impressive demo. Stick to the roadmap order — the lobby presence channel and team balancing algorithm are the two most technically differentiated features and should be working by Day 3.